

INTERNATIONAL UNIVERSITY OF RABAT
School of Digital Engineering & Artificial Intelligence

CV SCREENING MULTI-AGENT SYSTEM

Integrated AI/ML Software Engineering Project

Prepared by:
Zouhair Eouichak
Aymane El Youssi
Malak Bartal

Academic Year 2026 – 2027

Abstract

This report presents the design, implementation, and evaluation of a CV Screening Multi-Agent System developed as an Integrated AI/ML Software Engineering project. The system assists recruiters in efficiently screening and shortlisting job candidates by combining multiple specialist AI agents, a trained deep learning model, and a human-in-the-loop governance mechanism.

The architecture consists of three agents — a Technical Matcher, a Profile Analyzer, and an Orchestrator — working together to evaluate candidate CVs against job descriptions. A PyTorch-based classifier provides quantitative fit scoring, while a skill extraction tool performs structured skill-gap analysis. Uncertain or borderline cases are escalated to a human reviewer before a final decision is made.

The system achieves a pipeline accuracy of 83.33% on labeled evaluation cases, with the deep learning model reaching 100% accuracy on its test split. An interactive Streamlit frontend is provided for both single-candidate and batch screening workflows. All agent actions are logged to structured JSONL audit files for traceability and reproducibility.

Table of Contents

1. Introduction and Problem Statement
2. System Architecture
3. Methodology and Tools
4. Deep Learning Model
5. Frontend Demonstration
6. Evaluation and Testing
7. Results and Discussion
8. Challenges and Solutions
9. Conclusion and Future Work
10. Appendix

1. Introduction and Problem Statement

1.1 Context

In modern organizations, HR teams and recruiters receive dozens to hundreds of applications for each open position. Manually reviewing every CV is time-consuming, error-prone, and inconsistent. Decisions made under time pressure may overlook qualified candidates or introduce unconscious bias.

This project addresses this challenge by building an intelligent CV screening assistant that automates the initial evaluation of candidate-job fit while keeping human judgment available at critical decision points.

1.2 Project Goal

The goal is to design and implement a working multi-agent AI pipeline that:

- Separates technical and profile assessment into dedicated specialist agents
- Uses a trained PyTorch classifier as a scoring tool integrated into the agent workflow
- Provides explainable, intermediate outputs (matched skills, missing skills, rationales)
- Escalates uncertain or conflicting cases to a human reviewer before finalizing decisions
- Logs every decision step for audit, reproducibility, and debugging
- Offers an interactive frontend for live demonstration and practical use

1.3 Use Case

The system is designed around a concrete HR scenario: screening candidates for a Junior Data Analyst position. A recruiter can submit one CV at a time for detailed review, or submit a batch of CVs to obtain a ranked shortlist with downloadable reports.

2. System Architecture

2.1 Overview

The system follows a multi-agent architecture composed of two specialist agents and one orchestrator. Each agent has a defined role, inputs, and outputs. The orchestrator combines specialist results and enforces the human review policy.

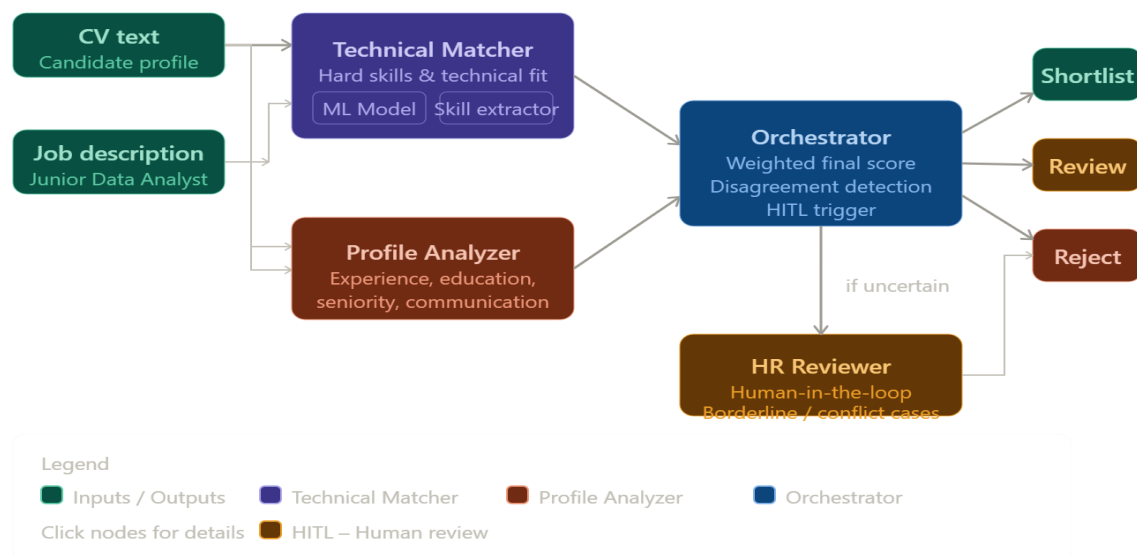


Figure 1: Architecture du système multi-agent

2.2 Agent Descriptions

Technical Matcher Agent

The Technical Matcher agent is responsible for evaluating the hard-skill and technical alignment between the candidate CV and the job description.

- Uses the PyTorch model tool to produce a fit probability score
- Uses the Skill Extractor tool to identify matched skills, missing required skills, and skill coverage ratio
- Outputs: a technical score (0–1), a fit label (Low / Medium / High), a rationale text, and structured skill gap data

Profile Analyzer Agent

The Profile Analyzer agent examines the non-technical dimensions of the candidate profile.

- Evaluates seniority signals, years of experience, project ownership, teamwork indicators, and education markers

- Produces a profile score (0–1), a fit label, and a written rationale

Orchestrator Agent

The Orchestrator combines the outputs from both specialist agents to produce the final hiring recommendation.

- Computes a weighted final score from technical and profile scores
- Maps the final score to one of three labels: High, Medium, or Low
- Maps labels to recommendations: Shortlist, Review, or Reject
- Detects specialist disagreement or borderline confidence and triggers the Human-in-the-Loop checkpoint when needed
- Optionally enriches the recommendation with a short professional summary via an Ollama local LLM

2.3 Human-in-the-Loop (HITL) Policy

Human review is triggered under any of the following conditions:

- The final score falls within the borderline range (neither clearly high nor clearly low)
- The two specialist agents strongly disagree on the fit assessment
- Any agent encounters an error or falls back to a heuristic due to a runtime failure

Two HITL modes are supported:

- CLI strict mode (`--require-human-approval`): marks pending review cases; does not auto-approve
- Streamlit single-screening mode: presents a blocking HR decision step (Shortlist / Reject / Needs Review) before the final recommendation is finalized

2.4 Runtime Backends

The system supports three runtime execution modes:

Runtime	Description	Availability
CrewAI	Primary agent framework; requires Python 3.11/3.12 and Ollama	Local environment
Deterministic	Fallback orchestrator; no LLM dependency; always available	Local + Colab
LangGraph	Stretch-goal scaffold for graph-based orchestration	Partial / optional

3. Methodology and Tools

3.1 Agent Tools

DL Model Tool (`model_tool.py`)

The Model Tool wraps the trained PyTorch classifier and exposes it as a callable agent tool.

- Input: CV text and job description text
- Output: fit label (Low / Medium / High), probability score, feature vector, matched skills, missing skills, and model source indicator
- Fallback: if the model file is missing or incompatible, the tool falls back to a deterministic heuristic instead of raising an error

Skill Extractor Tool (`skill_extractor_tool.py`)

The Skill Extractor tool performs keyword-based skill matching between the CV and the job description.

- Input: CV text and job description text
- Output: list of matched skills, list of missing required skills, full job skill list, and skill coverage ratio (0–1)

3.2 Technology Stack

Component	Technology / Library
Agent Framework	CrewAI (primary), Deterministic fallback, LangGraph scaffold
Deep Learning	PyTorch
LLM Backend	Ollama (llama3.2, local inference)
Frontend	Streamlit
Testing	pytest
Logging	Structured JSONL
Language	Python 3.11 / 3.12

3.3 Decision Flow

The pipeline follows a sequential, modular decision flow:

- Step 1: Parse and validate the input CV and job description
- Step 2: Technical Matcher runs the Model Tool and Skill Extractor Tool
- Step 3: Profile Analyzer scores experience, seniority, and profile quality
- Step 4: Orchestrator computes the weighted final score and determines the label
- Step 5: HITL checkpoint is evaluated; human input is requested if conditions are met
- Step 6: Optional LLM enrichment generates a professional recommendation summary
- Step 7: Final JSON result and JSONL audit log are written to disk

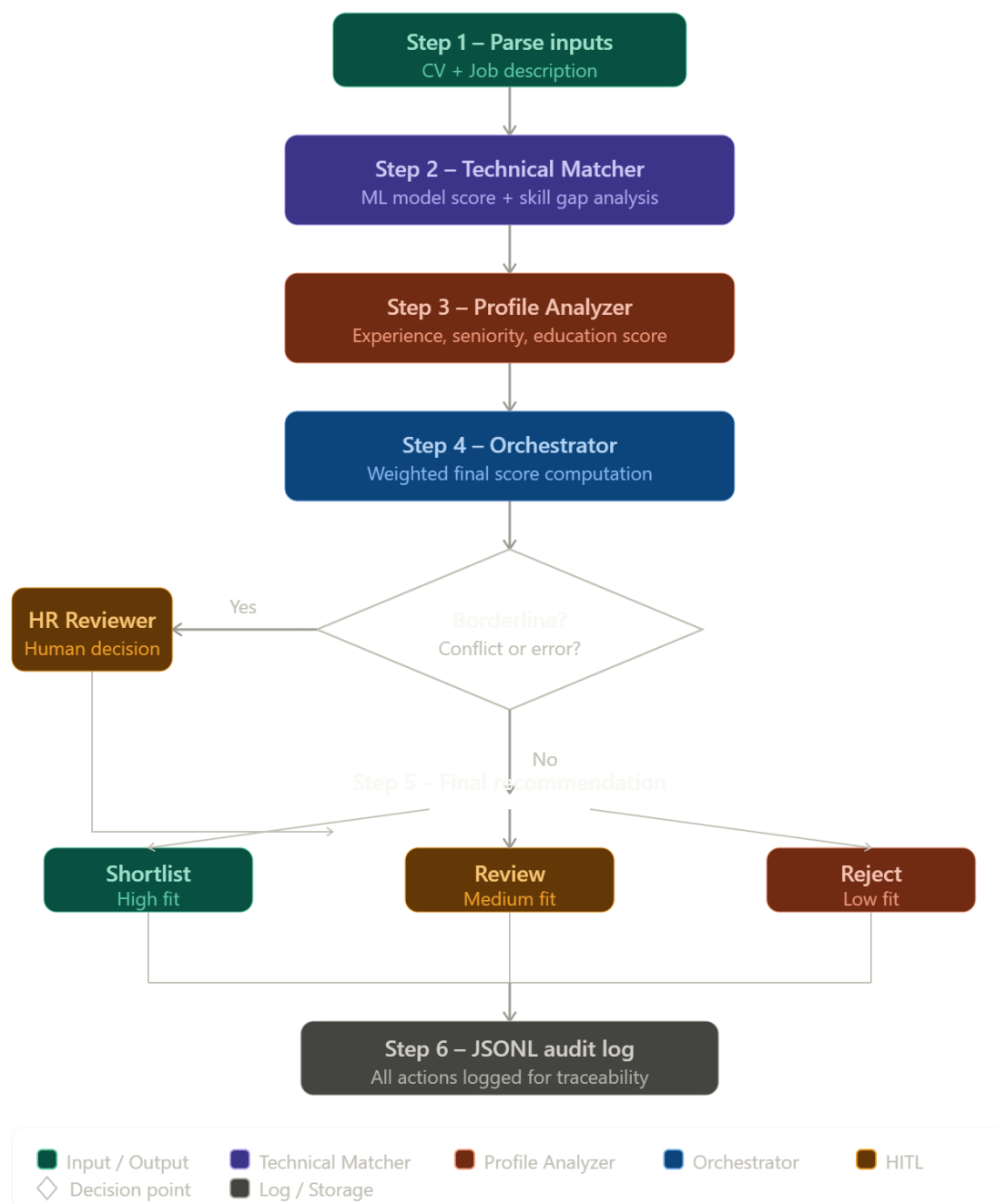


Figure 2: Pipeline Decision Flow Diagram

4. Deep Learning Model

4.1 Model Design

The project includes a custom PyTorch-based multi-class classifier designed to predict the CV-job fit level. The model is trained on a synthetically generated dataset of CV and job description feature vectors.

- Framework: PyTorch
- Task: multi-class classification (Low / Medium / High fit)
- Architecture: feedforward neural network with dropout regularization

- Training script: src/train_baseline.py
- Artifact: models/cv_fit_model.pt

4.2 Model Evaluation Results

The model was evaluated on a held-out test split of 60 labeled samples. The following metrics were recorded:

Metric	Value
Overall Accuracy	100%
Macro F1 Score	1.0
Precision – Low	1.0
Recall – Low	1.0
Precision – Medium	1.0
Recall – Medium	1.0
Precision – High	1.0
Recall – High	1.0
Test Set Size	60 samples

The model achieves perfect classification on the evaluation split, demonstrating clean separability between the three fit categories within the synthetic dataset.

4.3 Confusion Matrix

The confusion matrix confirms zero misclassifications across all three classes on the evaluation set:

- Low class: 24/24 correctly classified
- Medium class: 23/23 correctly classified
- High class: 13/13 correctly classified

```

(.venv) PS C:\Users\msi\Desktop\cv-screening-multiagent-main\cv-screening-multiagent-main> python -m src.train_baseline
Generating synthetic dataset (300 samples)...
Saved synthetic dataset to C:\Users\msi\Desktop\cv-screening-multiagent-main\cv-screening-multiagent-main\data\train_synthetic.json
Epoch 10: loss=0.2620 val_acc=0.9833 val_f1=0.9833
Epoch 20: loss=0.0615 val_acc=1.0000 val_f1=1.0000
Epoch 30: loss=0.0294 val_acc=1.0000 val_f1=1.0000
Epoch 40: loss=0.0171 val_acc=1.0000 val_f1=1.0000
Epoch 50: loss=0.0111 val_acc=1.0000 val_f1=1.0000
Epoch 60: loss=0.0077 val_acc=1.0000 val_f1=1.0000
Epoch 70: loss=0.0061 val_acc=1.0000 val_f1=1.0000
Epoch 80: loss=0.0047 val_acc=1.0000 val_f1=1.0000
Epoch 90: loss=0.0037 val_acc=1.0000 val_f1=1.0000
Epoch 100: loss=0.0030 val_acc=1.0000 val_f1=1.0000
Epoch 110: loss=0.0024 val_acc=1.0000 val_f1=1.0000
Epoch 120: loss=0.0022 val_acc=1.0000 val_f1=1.0000
Epoch 130: loss=0.0018 val_acc=1.0000 val_f1=1.0000
Epoch 140: loss=0.0017 val_acc=1.0000 val_f1=1.0000
Epoch 150: loss=0.0015 val_acc=1.0000 val_f1=1.0000

=== Test Set Evaluation ===
Accuracy : 1.0000
Macro-F1 : 1.0000
Low      : P=1.000 R=1.000 F1=1.000
Medium   : P=1.000 R=1.000 F1=1.000
High     : P=1.000 R=1.000 F1=1.000
Confusion matrix (rows=true, cols=pred):
          Low      Med      High
Low      : [24, 0, 0]
Medium   : [0, 23, 0]
High     : [0, 0, 13]

Model saved to: C:\Users\msi\Desktop\cv-screening-multiagent-main\cv-screening-multiagent-main\models\cv_fit_model.pt
Training log saved to: C:\Users\msi\Desktop\cv-screening-multiagent-main\cv-screening-multiagent-main\logs\train_20260524T101328Z.json
Evaluation summary saved to: C:\Users\msi\Desktop\cv-screening-multiagent-main\cv-screening-multiagent-main\models\model_evaluation.json
Evaluation markdown saved to: C:\Users\msi\Desktop\cv-screening-multiagent-main\cv-screening-multiagent-main\models\model_evaluation.md

```

Figure 3: PyTorch Model Confusion Matrix on 60-sample test set

4.4 Integration with the Agent Workflow

The trained model is loaded by the Model Tool (`src/tools/model_tool.py`) at runtime. If the model artifact is available, it is used for inference; otherwise, the tool falls back to a deterministic heuristic to ensure the pipeline never crashes due to a missing model file.

5. Frontend Demonstration

5.1 Streamlit Application

The project includes a full-featured Streamlit web application (src/app_frontend.py) designed for live presentation and practical recruiter use. The interface provides two primary screening modes and several supporting features.

5.2 Single Screening Mode

The Single Screening mode allows a recruiter to evaluate one candidate against the Junior Data Analyst job description.

- Supports PDF CV upload with automatic text extraction
- Displays the final recommendation label and score
- Shows both agent outputs (technical and profile rationales)
- Presents a structured skill gap panel (matched skills vs. missing skills)
- Includes explainability highlights for the decision
- Offers a blocking HR review step (Shortlist / Reject / Needs Review) when the case is borderline

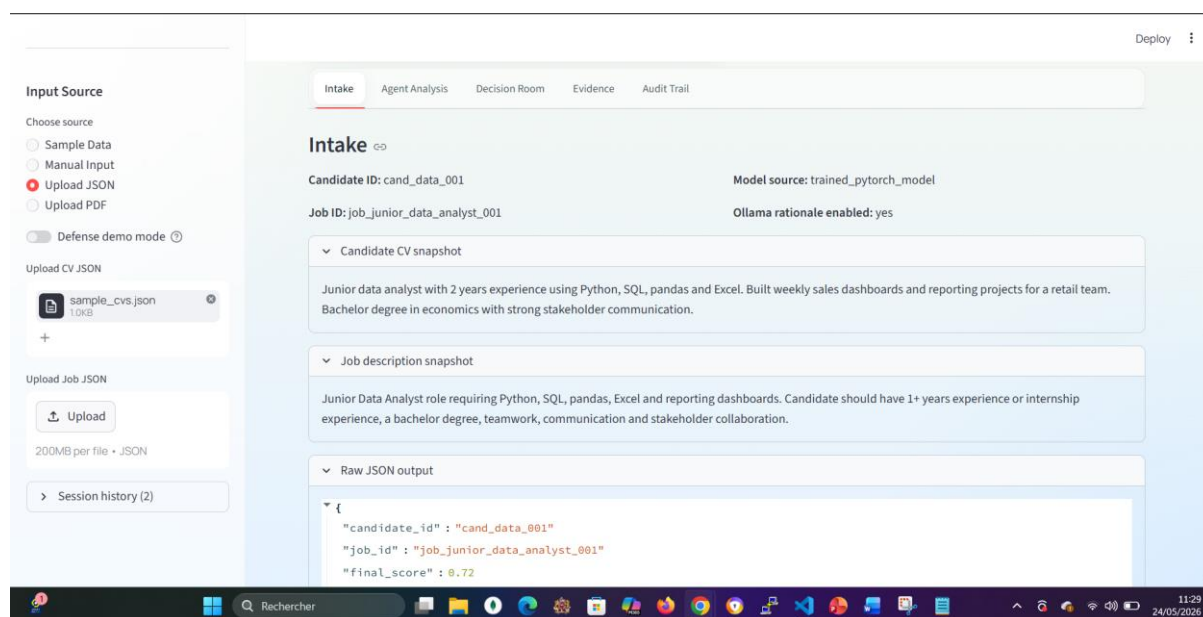


Figure 4: Single Screening Mode: Intake tab showing CV and job description

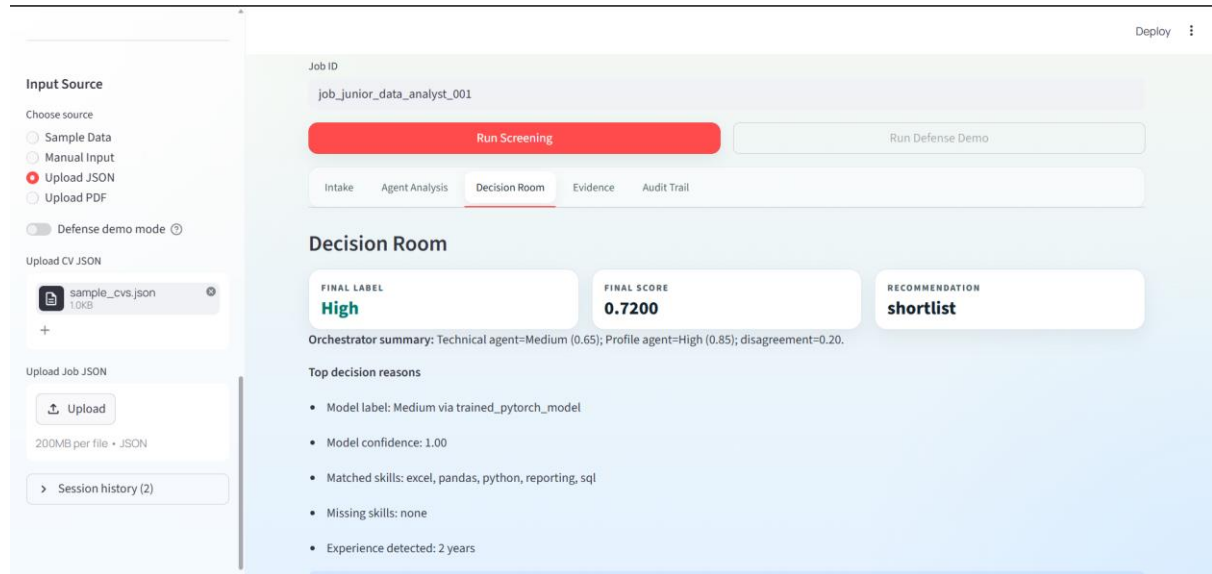


Figure 5: Single Screening Mode: Decision Room with final label and skill gap

5.3 Batch Screening Mode

The Batch Screening mode processes multiple CVs against the same job description simultaneously.

- Produces a ranked leaderboard of all candidates
- Shows final labels, scores, and recommendations for each candidate
- Flags borderline cases for HR review in the table and audit log
- Provides downloadable JSON and CSV export buttons for the full results

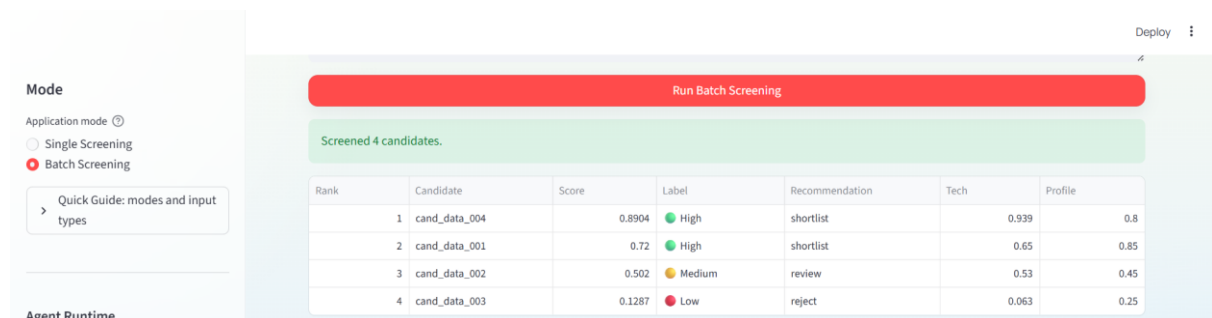


Figure 6: Batch Screening Mode: candidate leaderboard ranked by final score

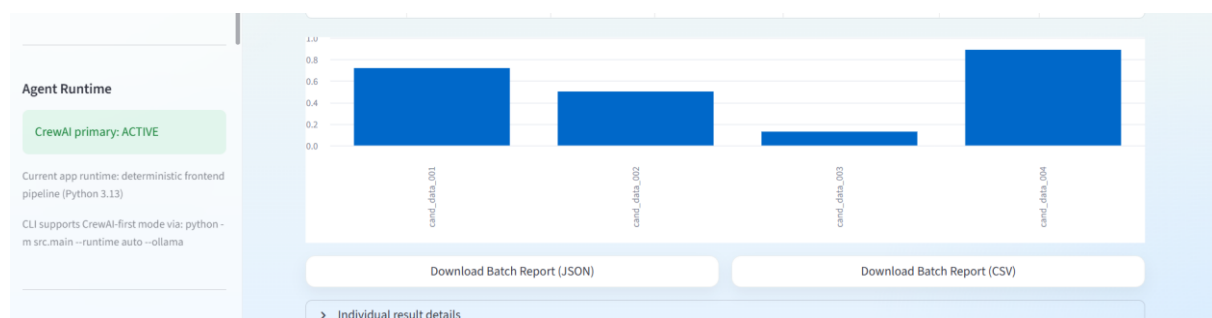


Figure 7: Batch Screening Mode: bar chart and downloadable reports

5.4 Additional Features

- Session history panel for reviewing past screenings in the same session
- Defense checklist panel showing alignment evidence for each project requirement
- Showcase metrics panel displaying model evaluation and pipeline evaluation summaries
- Ollama server and model controls in the sidebar for managing the local LLM backend

6. Evaluation and Testing

6.1 Automated Test Suite

The project includes a pytest test suite covering pipeline smoke tests, edge cases, batch screening logic, and frontend helper functions.

Test Module	Test Cases	Status
test_batch_screening.py	Batch screening flow	Pass
test_edge_cases.py	Edge cases and fallbacks	Pass
test_frontend_batch_helpers.py	Frontend helper functions	Pass
test_pipeline_smoke.py	End-to-end smoke test	Pass

All 16 tests pass successfully across the four test modules.

6.2 Pipeline Evaluation on Labeled Cases

The pipeline was evaluated on 6 labeled CV screening cases with known expected fit labels. The following results were recorded:

Case ID	Expected	Predicted	Score	Recommendation
eval_001	High	High ✓	0.982	Shortlist
eval_002	Medium	Medium ✓	0.607	Review
eval_003	High	Medium ✗	0.627	Review
eval_004	Medium	Medium ✓	0.479	Review (borderline)
eval_005	Low	Low ✓	0.252	Reject
eval_006	Low	Low ✓	0.371	Reject

Pipeline Metric	Value
Overall Accuracy	83.33% (5/6 correct)
Macro Precision	88.89%
Macro Recall	83.33%
Macro F1 Score	82.22%

The single misclassification (eval_003) involves a borderline High candidate scored as Medium by the pipeline. This case demonstrates how the HITL checkpoint serves as a safety net: borderline cases are flagged for human review rather than silently accepted as final decisions.

6.3 Logging and Traceability

Every agent action is logged to a structured JSONL file. Each log event includes:

- timestamp, agent_name, action, tool_used
- input_summary, output_summary, status, error
- event type and full payload

The screenshot displays the 'Audit Trail' section of the CV Screening Multi-Agent System. On the left, a sidebar contains 'Mode' (Application mode: Single Screening, Batch Screening), 'Agent Runtime' (CrewAI primary: ACTIVE), and 'LLM Backend'. The main panel shows a list of log events from a batch screening run. The events are as follows:

Timestamp	Agent Name	Action	Status
2026-05-24T10:35:38.034028+00:00	orchestrator	complete_batch_screening	SUCCESS
2026-05-24T10:35:38.026127+00:00	orchestrator	complete_screening	SUCCESS
2026-05-24T10:35:38.018722+00:00	orchestrator	final_decision	SUCCESS
2026-05-24T10:35:38.008551+00:00	orchestrator	generate_llm_rationale	FAILURE
2026-05-24T10:35:35.964545+00:00	profile_analyzer	produce_assessment	SUCCESS
2026-05-24T10:35:35.956390+00:00	technical_matcher	produce_assessment	SUCCESS
2026-05-24T10:35:35.928194+00:00	orchestrator	start_screening	STARTED

Figure 8: Audit Trail: structured JSONL log events from a batch screening run

7. Results and Discussion

7.1 Summary of Results

The CV Screening Multi-Agent System successfully delivers on all core project requirements. The pipeline produces consistent, explainable recommendations while maintaining human oversight for uncertain decisions.

Key quantitative results:

- PyTorch model test accuracy: 100% on 60 labeled samples
- End-to-end pipeline accuracy: 83.33% on 6 labeled evaluation cases
- All 16 automated tests passing
- Full Streamlit frontend operational with single and batch screening modes

7.2 Strengths

- Modular architecture allows each agent to be tested and replaced independently
- Dual runtime support (CrewAI + deterministic fallback) ensures the system never fails to produce a result
- Explainability features (skill gap panel, rationale text) make decisions transparent to recruiters
- HITL mechanism prevents automated decisions on uncertain cases without human review
- Structured JSONL logging enables full audit trails and debugging

7.3 Limitations

- The PyTorch model was trained on synthetic data and may not generalize directly to real-world CVs without retraining on real annotated datasets
- Skill extraction is keyword-based and does not capture semantic equivalences (e.g., 'Pandas' vs. 'data manipulation library')
- Ollama LLM inference speed depends heavily on local hardware and can introduce latency
- Pipeline accuracy is measured on a small 6-case evaluation set; a larger benchmark is needed for robust statistical conclusions

8. Challenges and Solutions

Challenge	Root Cause	Solution Implemented
Long Ollama response time during demos	LLM generation latency and occasional startup delay	Added server readiness checks, model pull verification, and reduced max token length for faster responses
Streamlit blocked by CLI human input prompt	Borderline cases triggered a terminal input() call inside a web flow	Added explicit blocking HR decision UI in Single Screening and preserved non-blocking batch mode
Port conflicts during demo startup	Stale Streamlit processes occupying the default port	Added controlled startup flow in run_project.ps1 with configurable alternative port
CrewAI incompatibility with Python 3.14+	CrewAI requires Python < 3.14; newer Python versions break the dependency	Documented Python 3.11/3.12 as recommended runtime; deterministic fallback ensures the pipeline runs on any version

9. Conclusion and Future Work

9.1 Conclusion

This project successfully implements a production-oriented CV screening assistant that combines multi-agent reasoning, deep learning scoring, structured explainability, and human oversight. The system is reproducible, test-validated, and ready for live demonstration.

The architecture demonstrates how multiple specialized AI agents, each with well-defined tools and decision responsibilities, can be composed into a reliable and auditable pipeline. The HITL policy ensures that the system does not make autonomous decisions when confidence is low, which is critical for any production HR application.

9.2 Future Work

- Replace keyword-based skill extraction with semantic embedding models (e.g., sentence transformers) to capture conceptual skill equivalences
- Retrain the PyTorch model on real-world annotated CV datasets to improve generalization and robustness
- Add fairness and bias metrics to detect and mitigate demographic disparities in recommendations
- Implement a recruiter feedback loop for continuous model improvement based on actual hiring outcomes
- Add dashboard analytics for visualizing screening trends and pipeline performance over time
- Extend the system to support multilingual CV and job description processing
- Explore more advanced LangGraph-based orchestration for dynamic agent retry and branching workflows

10. Appendix

10.1 Project Structure

The repository is organized as follows:

- data/ – Sample input CVs, job descriptions, and labeled evaluation cases
- docs/ – Architecture documentation and setup guides
- logs/ – Runtime JSONL logs and pipeline evaluation summary
- models/ – Trained PyTorch model artifact and evaluation metrics
- src/agents/ – Technical Matcher, Profile Analyzer, and Orchestrator agent code
- src/pipeline/ – Batch screening, scoring, and HITL checkpoint logic
- src/tools/ – Model tool and skill extractor tool
- src/utis/ – Shared JSON logger utility
- tests/ – Smoke tests, edge case tests, and batch helper tests
- README.md – Full setup and run instructions

10.2 How to Reproduce

- Step 1: Create and activate a Python 3.11 or 3.12 virtual environment
- Step 2: Install dependencies with: `pip install -r requirements.txt`
- Step 3: Train the model with: `python -m src.train_baseline`
- Step 4: Run the pipeline with: `python -m src.main`
- Step 5: Launch the frontend with: `streamlit run src/app_frontend.py`
- Step 6 (Windows demo): use `./run_project.ps1` for one-command setup and launch

10.3 Additional Screenshots

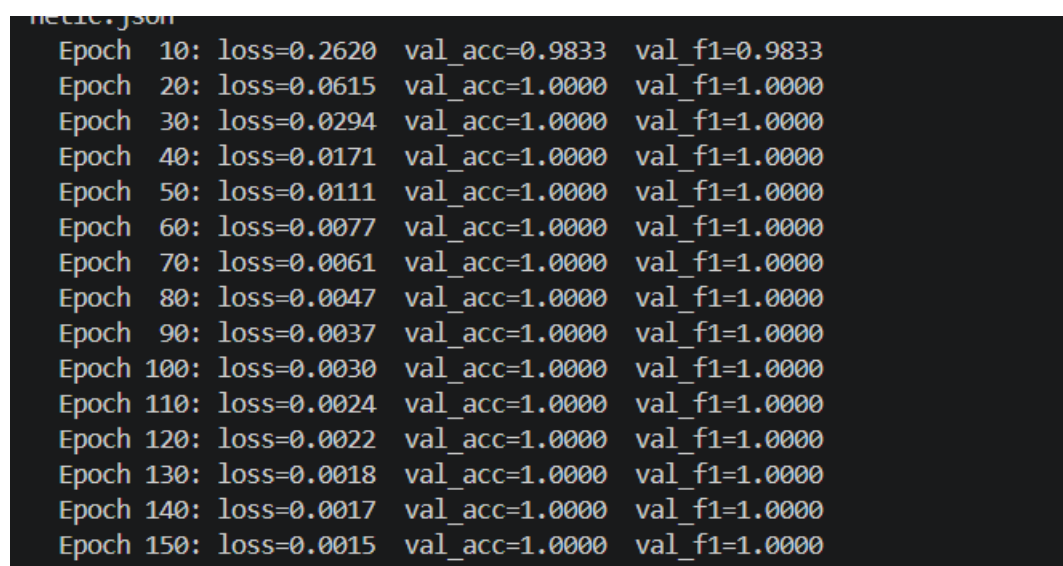


Figure 9: Model Training Output: loss and accuracy across 150 epochs

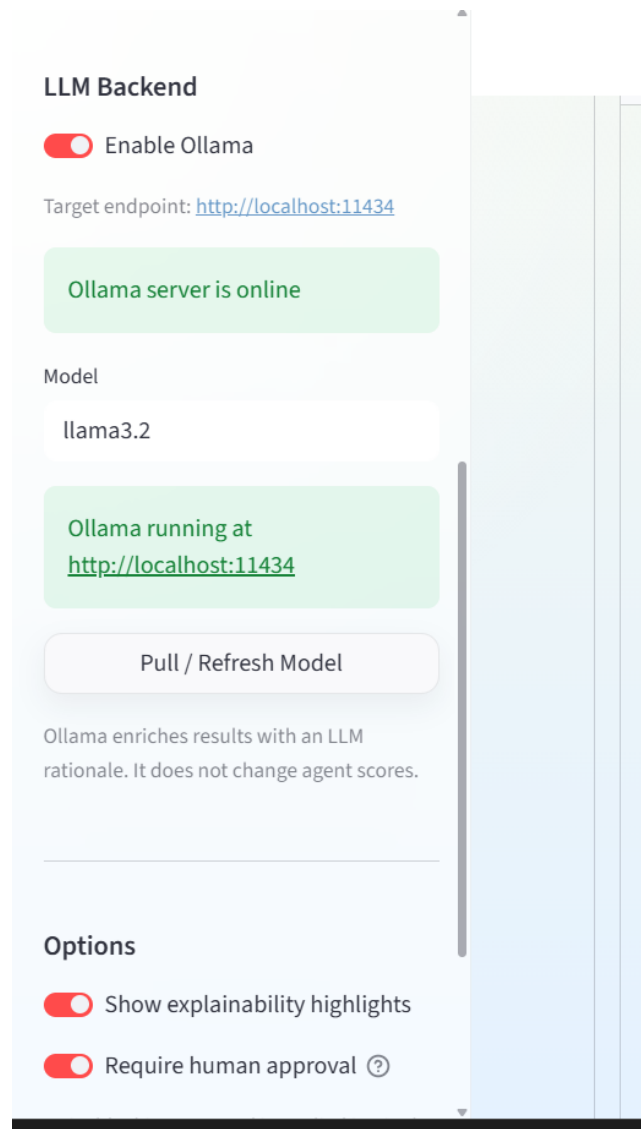


Figure 10: Sidebar Controls: Ollama LLM backend and options panel

```
(.env) PS C:\Users\msi\Desktop\cv-screening-multiagent-main\cv-screening-multiagent-main> cat logs/run_20260524T102601Z.jsonl
{"timestamp": "2026-05-24T10:26:01.579748+00:00", "agent_name": "orchestrator", "action": "start_screening", "tool_used": "technical_matcher,profile_analyzer", "input_summary": "candidate_id=cand_data_001, job_id=job_junior_data_analyst_001", "output_summary": null, "status": "started", "error": null, "event": "screening_started", "payload": {"candidate_id": "cand_data_001", "job_id": "job_junior_data_analyst_001"}}
{"timestamp": "2026-05-24T10:26:01.601631+00:00", "agent_name": "technical_matcher", "action": "produce_assessment", "tool_used": "model_tool,skill_extractor_tool", "input_summary": "cv_text + job_text", "output_summary": "label=Medium, score=0.6501, recommendation=review", "status": "success", "error": null, "event": "agent_result", "payload": {"agent_name": "technical_matcher", "score": 0.6500699999999999, "label": "Medium", "rationale": "Model=Medium(0.50), skills_coverage=1.00, matched=['excel', 'pandas', 'python', 'reporting', 'sql'], missing=[], 'success': true, 'error': null, 'recommendation': 'review', 'evidence': ['Model label: Medium via trained_pytorch_model', 'Model confidence: 1.00', 'Matched skills: excel, pandas, python, reporting, sql', 'Missing skills: none'], 'metadata': {'model_used': 'trained_pytorch_model', 'model_confidence': 0.9998, 'feature_vector': [1.0, 0.5184999704360962, 1.0, 0.0, 1.0, 1.0, 0.0, 1.0], 'matched_skills': ['excel', 'pandas', 'python', 'reporting', 'sql'], 'missing_skills': []}}
{"timestamp": "2026-05-24T10:26:01.605193+00:00", "agent_name": "profile_analyzer", "action": "produce_assessment", "tool_used": "heuristic_profile_rules", "input_summary": "cv_text + job_text", "output_summary": "label=High, score=0.8500, recommendation=shortlist", "status": "success", "error": null, "event": "agent_result", "payload": {"agent_name": "profile_analyzer", "score": 0.85, "label": "High", "rationale": "Profile score based on years alignment (1.00), leadership=False, project_ownership=True, communication=True, education=True.", "success": true, "error": null, "recommendation": "shortlist", "evidence": ["Experience detected: 2 years", "Project ownership or delivery evidence present", "Communication or collaboration signals present", "Formal degree signal detected"], "metadata": {"cv_years": 2, "job_years": 1, "leadership": false, "project_ownership": true, "communication": true, "education": true}}}
{"timestamp": "2026-05-24T10:26:03.672508+00:00", "agent_name": "orchestrator", "action": "generate_llm_rationale", "tool_used": "ollama_generate", "input_summary": null, "output_summary": null, "status": "failure", "error": "ollama unreachable at http://localhost:11434. Is 'ollama serve' running?", "event": "llm_rationale_error", "payload": {"error": "ollama unreachable at http://localhost:11434. Is 'ollama serve' running?"}}
{"timestamp": "2026-05-24T10:26:03.680976+00:00", "agent_name": "orchestrator", "action": "final_decision", "tool_used": "decision_policy", "input_summary": "tech_score=0.6501, profile_score=0.8500", "output_summary": "final_label=High, recommendation=shortlist", "status": "success", "error": null, "event": "orchestrator_decision", "payload": {"candidate_id": "cand_data_001", "job_id": "job_junior_data_analyst_001", "final_label": "High", "final_recommendation": "shortlist", "final_score": 0.75}}
```

Figure 11:Raw JSONL Audit Log: full structured event trace

```
(.env) PS C:\Users\msi\Desktop\cv-screening-multiagent-main\cv-screening-multiagent-main> python -m src.preflight_demo
=====
PRECHECK FAIL
=====
[PASS] python: Python 3.13
[PASS] crewai: CrewAI installed
[PASS] streamlit: Streamlit installed
[PASS] trained_model: Model artifact found at C:\Users\msi\Desktop\cv-screening-multiagent-main\cv-screening-multiagent-main\models\cv_fit_model.pt
```

Figure 12: Preflight Demo: system readiness check output